K8S-99: Best Practice Summary

> **Series:** K8S — Kubernetes Monitoring | **Notebook:** 99 | **Created:** March 2026 | **Last Updated:** 10/22/2026

Overview

This notebook consolidates every actionable best practice for Dynatrace Kubernetes monitoring and DynaKube configuration extracted from the K8S series (notebooks 01-14). Each practice specifies the exact setting, value, priority, and category. Use this as a definitive checklist for new deployments and audits of existing environments.

Operator Version: 1.10.2 (validated pin in this series; 1.11.0, released 10/01/2026, is the newest — see the note below) | **DynaKube API:** `dynatrace.com/v1beta6` — current, use for new DynaKubes (`v1beta5` remains accepted; no rewrite required) | **Helm:** `oci://public.ecr.aws/dynatrace/dynatrace-operator --version 1.10.2` — skip 1.10.0 (pin the version you have validated in your estate)

> **Operator version policy (as of 08/11/2026). 1.10.2 is the recommendation** — released July 30, 2026 with a published changelog. It resolves four issues: a workload/namespace **tagging-precedence regression**, a `dynatrace-webhook` `CrashLoopBackOff` under the gVisor runtime class, injected pods hanging on the OneAgent-binary download (timeout raised to 15 minutes), and metadata-enrichment rules that could not be applied being silently disregarded rather than logged.
>
> **The tagging fix is a behaviour change, not just a bug fix.** Where several tagging rules match the same key, 1.10.2 applies **only the first matching rule**; earlier versions let each subsequent rule overwrite the previous one. A cluster whose enrichment depends on last-rule-wins ordering will produce different tags after the upgrade — audit your rules first. See K8S-10.
>
> **1.10.1 remains a working pin.** Estates upgrade operators on their own schedule, and 1.10.1 is still the fix for the 1.10.0 defects — subject to the OpenShift-manifest caveat in K8S-09 §2. **Skip 1.10.0** — its own release notes advise skipping it (auto-update defect) and it is flagged `prerelease: true` on the GitHub releases page, a signal you can verify before pinning. Do not pin below 1.4.1 in any case (CSI liveness-probe crash-loop window — K8S-09 §2).
>
> **Operator 1.11.0 (released 10/01/2026).** Removes the `v1beta4` DynaKube API from the CRD (*"Applying DynaKube resources that still use v1beta4 will fail"*), adds image-volume code-module injection, which *"replaces the CSI driver as the recommended approach"* (Kubernetes 1.35+ and containerd 2.2+ or CRI-O 1.33+ required; rule 7), adds `spec.kubernetesMonitoring` to size the Kubernetes-monitoring ActiveGate separately in one DynaKube, and accepts workload and pod labels and annotations as enrichment sources (rules 39–41). Minimum version for a direct upgrade: 1.6.0. Move the pin once you have validated 1.11.0 on a non-production cluster.
>
> **Token currency note:** Platform Tokens (`dt0s16`) are the recommended choice for new tenants per Dynatrace SaaS sprint-1.337+; the Operator itself accepts platform tokens from **Operator 1.10.0** (July 15, 2026) — Classic API Tokens (`dt0c01`) continue to be accepted and remain the working path on earlier Operator versions. See K8S-02 §Prerequisites.
>
> **Operator 1.10.0 upgrade notes (July 15, 2026):** the ActiveGate `TopologySpreadConstraint` default changed from `DoNotSchedule` to `ScheduleAnyway` — expect an **ActiveGate restart on upgrade** (an explicitly-set `whenUnsatisfiable` wins, so the recommended posture is to set it; see K8S-02 §8). DynaKube `v1beta3` was removed in Operator 1.9.0, and **`v1beta4` stopped being served in 1.10.0** (it was deprecated in 1.9.0) — so a `v1beta4` manifest fails to apply after this upgrade, not merely warns. `v1beta5` is still served but is itself flagged deprecated from 1.10.0. Target `v1beta6` for new DynaKubes. The `gke-autopilot.yaml` release artifact is removed — GKE Autopilot deploys via Helm. Two init containers were introduced — **`webhook-cert-generator`** and **`crd-storage-migrator`** — which is what an `Init:0/1` hang is waiting on (K8S-09 §2). 1.10.0 also adds a **CSI-to-ephemeral-volume migration mode**, which turns code-module delivery into a deliberate choice rather than a fixed default — see rule 7 and K8S-12 §2.

Table of Contents

1. Deployment Mode Selection

2. Operator Installation
3. DynaKube Core Configuration
4. ActiveGate Configuration
5. Namespace and Injection Control
6. Feature Flags and Annotations
7. Metadata Enrichment
8. Log Monitoring
9. OTel Collector and Telemetry Ingest
10. CSI Driver Configuration
11. Resource Sizing
12. Security and Secrets
13. GitOps and Lifecycle
14. Cluster Health Alerting
15. Workload and Application Monitoring
16. Specialized Monitoring
17. Troubleshooting Practices

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Grail and Kubernetes monitoring enabled
Kubernetes Cluster	A version supported by your Operator release
Helm	v3.x
Dynatrace Operator	v1.10.2 (July 30, 2026, recommended pin) via `oci://public.ecr.aws/dynatrace/dynatrace-operator` — v1.10.1 remains a working pin until you upgrade; skip 1.10.0 (see the version-policy note above)
Knowledge	K8S-01 through K8S-13

1. Deployment Mode Selection

#	Best Practice	Recommended Setting/Value	Priority	Category
1	Use `cloudNativeFullStack` for new deployments	`spec.oneAgent.cloudNativeFullStack: {}`	**Critical**	Deployment
2	Do NOT use `classicFullStack` for new deployments	Remove `spec.oneAgent.classicFullStack`	**Critical**	Deployment
3	Use `applicationMonitoring` when infra visibility is not needed	`spec.oneAgent.applicationMonitoring: {}` (CSI driver use is chosen at Operator install, not in the DynaKube)	Recommended	Deployment
4	Use `hostMonitoring` for infra-only clusters	`spec.oneAgent.hostMonitoring: {}`	Optional	Deployment
5	Omit `oneAgent` entirely for infrastructure-only monitoring alongside other APM tools	Only configure `spec.activeGate`	Recommended	Deployment

> **classicFullStack** is legacy. It uses host-path mounts and shares a single OneAgent across all pods. `cloudNativeFullStack` injects code modules via the webhook — delivered by image volume, CSI driver or ephemeral volume (rule 7) — enabling

> independent app/infra monitoring with no privileged containers for applications.

2. Operator Installation

#	Best Practice	Recommended Setting/Value	Priority	Category
6	Install operator via Helm OCI, always with an explicit `--version`	`helm upgrade dynatrace-operator oci://public.ecr.aws/dynatrace/dynatrace-operator --version 1.10.2 --namespace dynatrace --create-namespace --install --atomic`	**Critical**	Installation
7	Choose a code-module delivery mode deliberately	Image volumes on Operator 1.11.0+ where nodes meet the requirements (Dynatrace's recommended approach); otherwise `csidriver.enabled: true`; ephemeral volumes when a privileged DaemonSet is ruled out (Operator 1.10.0+). To leave CSI, migrate first — never flip `csidriver.enabled: false` first	**Critical** *(making the choice)*	Installation
8	Set platform explicitly	`platform: "kubernetes"` or `platform: "openshift"`	Recommended	Installation
9	Create dedicated namespace	`kubectl create namespace dynatrace`	**Critical**	Installation
10	Create API token secret before applying DynaKube	`kubectl create secret generic dynakube --namespace dynatrace --from-literal=apiToken=<TOKEN> --from-literal=dataIngestToken=<TOKEN>`	**Critical**	Installation

> **Rule 7 — code-module delivery is a choice, not an invariant.** The Helm chart turns the CSI driver on by default, but it has always been optional: the plain `kubernetes.yaml` manifest installs without it. **Operator 1.10.0 adds a CSI-to-ephemeral-volume migration mode** that moves an existing cluster off CSI in one restart cycle, and **Operator 1.11.0 adds image volumes**, which Dynatrace now recommends over the CSI driver. What is Critical is *making the decision consciously* — not one specific value.
>
> | Delivery mode | Choose it when | Cost |
> |---|---|---|
> | **Image volumes** (Operator 1.11.0+; DynaKube flag `feature.dynatrace.com/mount-code-modules-via-image-volume: "true"`) | Dynatrace's recommended approach where the cluster runs Kubernetes 1.35+ with containerd 2.2+ or CRI-O 1.33+. Not compatible with the built-in tenant registry. | Existing pods keep their mounts until restarted, so a full switch needs a rolling restart. |
> | **CSI driver** (`csidriver.enabled: true`) | The image-volume requirements are not met. You want code modules cached per node and shared across pods, and you can run a privileged DaemonSet with a host-path socket. | An extra 5-container DaemonSet to size and monitor; a mount-storm failure mode (rules 57–59, K8S-09 §2). |
> | **Ephemeral volumes** (no CSI driver: `csidriver.enabled: false`, or the plain `kubernetes.yaml` manifest) | A CSI DaemonSet is unacceptable — restrictive admission policy, a managed platform that limits CSI drivers, or a node pool where you will not run privileged workloads. | Code modules are copied per pod rather than shared per node — the docs call it *"storage-inefficient"* and budget 1 GB of ephemeral storage per monitored pod, against 0.1 GB with the CSI driver — so pod starts can slow at high density; measure on your busiest nodes. |
>
> Migrating CSI → ephemeral (the documented direction) is a Helm-values change plus one workload-restart cycle, not an application change: enable `csidriver.migrationMode`, restart injected workloads, confirm no pod still mounts `csi.oneagent.dynatrace.com`, and only then set `csidriver.enabled: false` — pods still on CSI mounts stop working once the driver is disabled. Migration mode needs Operator 1.10.0 (released July 15, 2026), and estates adopt it on their own schedule. On 1.9.x and earlier you can still run without CSI, but there is no migration mode, so moving an existing cluster off CSI takes longer — the how-to measures migration mode as *"a single pod-restart cycle instead of two"*. Mechanics and trade-offs: K8S-12 §2–§3.

> **Sources:** Operator Helm chart values, v1.11.0 (Dynatrace GitHub) — `csidriver.enabled: true`; Use image volumes for code modules injection (DT docs) — *"ephemeral volume injection needs no privileged access but is storage-inefficient"*; Storage requirements (DT docs) — the per-pod figures; Migrate from CSI driver to ephemeral volumes (DT docs). The `kubernetes.yaml` / `kubernetes-csi.yaml` split is read from the Operator release assets (Dynatrace GitHub), 10/02/2026.

Required Token Scopes

The docs describe platform tokens (Operator 1.10.0+); classic access tokens remain accepted. Full tables: K8S-02 § Prerequisites.

Operator token: `fleet-management:activegate.connection-info:read`, `fleet-management:activegate.tokens:create`, `fleet-management:container-images:read`, `fleet-management:oneagent.connection-info:read`, `fleet-management:oneagents:download`, `settings:objects:read`, `settings:objects:write`

Data Ingest token: `openpipeline:logs:ingest`, `openpipeline:metrics:ingest`, `openpipeline:traces:ingest`, `storage:metrics:write`

3. DynaKube Core Configuration

#	Best Practice	Recommended Setting/Value	Priority	Category
11	Use `v1beta6` for new DynaKubes	`apiVersion: dynatrace.com/v1beta6`. `v1beta5` still applies but is **deprecated from 1.10.0**; `v1beta4` is **not served from 1.10.0** and **removed from the CRD in 1.11.0** (apply fails); `v1beta3` **removed** in 1.9.0	**Critical**	Configuration
12	Set `apiUrl` to your SaaS tenant	`spec.apiUrl: https://ENVIRONMENT_ID.live.dynatrace.com/api`	**Critical**	Configuration
13	Add control-plane tolerations to OneAgent	`tolerations: [{effect: NoSchedule, key: node-role.kubernetes.io/master, operator: Exists}, {effect: NoSchedule, key: node-role.kubernetes.io/control-plane, operator: Exists}]`	Recommended	Configuration
14	Set `nodeSelector` for linux-only	`nodeSelector: {kubernetes.io/os: linux}`	Recommended	Configuration
15	Set OneAgent resource requests and limits	`spec.oneAgent.cloudNativeFullStack.oneAgentResources` — e.g. `requests: {cpu: 100m, memory: 256Mi}`, `limits: {cpu: 300m, memory: 512Mi}` as a community starting point; size from measured usage	Recommended	Configuration
16	Set `networkZone` for routing isolation	`spec.networkZone: production`	Optional	Configuration
17	Set `hostGroup` for logical grouping	`spec.oneAgent.hostGroup: production`	Optional	Configuration

Minimal Production DynaKube

```
apiVersion: dynatrace.com/v1beta6
kind: DynaKube
metadata:
  name: dynakube
  namespace: dynatrace
spec:
  apiUrl: https://ENVIRONMENT_ID.live.dynatrace.com/api
  metadataEnrichment:
    enabled: true
  oneAgent:
    cloudNativeFullStack:
      tolerations:
        - effect: NoSchedule
          key: node-role.kubernetes.io/master
          operator: Exists
        - effect: NoSchedule
          key: node-role.kubernetes.io/control-plane
          operator: Exists
  activeGate:
    capabilities:
      - kubernetes-monitoring
      - routing
      - dynatrace-api
```

4. ActiveGate Configuration

#	Best Practice	Recommended Setting/Value	Priority	Category
18	Enable `kubernetes-monitoring` capability	`spec.activeGate.capabilities: [kubernetes-monitoring, routing]`	**Critical**	ActiveGate
19	Add `dynatrace-api` capability	Add `dynatrace-api` to capabilities list	Recommended	ActiveGate
20	Set ActiveGate replicas >= 2 for production	`spec.activeGate.replicas: 2`	**Critical**	ActiveGate
21	Set ActiveGate resource limits	e.g. `limits: {cpu: 1000m, memory: 2Gi}` for medium clusters (community starting point; FAQ-10 for sizing)	**Critical**	ActiveGate
22	Add zone-aware topology spread	`topologySpreadConstraints: [{maxSkew: 1, topologyKey: topology.kubernetes.io/zone, whenUnsatisfiable: ScheduleAnyway}]`	Recommended	ActiveGate

ActiveGate Sizing Reference

Starting points from community practice, not Dynatrace-published figures — measure and adjust (FAQ-10).

Cluster Size	Nodes	CPU Limit	Memory Limit	Replicas
Small	1-10	500m	1Gi	1
Medium	10-50	1000m	2Gi	2
Large	50-100	2000m	4Gi	2
Enterprise	100+	4000m	8Gi	3+

5. Namespace and Injection Control

#	Best Practice	Recommended Setting/Value	Priority	Category
23	Use `namespaceSelector` to control injection scope	`spec.oneAgent.cloudNativeFullStack.namespaceSelector.matchLabels: {dynatrace-injection: enabled}`	Recommended	Injection
24	Do not use the `default` namespace for workloads	Enforce via policy	Recommended	Namespace
25	Apply consistent labels to all namespaces	`team`, `env`, `cost-center` labels	Recommended	Namespace
26	Use `matchExpressions` to exclude system namespaces	`operator: NotIn, values: [kube-system, newrelic, datadog]`	Recommended	Injection
27	Use pod annotation to disable injection for specific pods	`oneagent.dynatrace.com/inject: "false"`	Optional	Injection
28	Limit injected technologies when needed	`oneagent.dynatrace.com/technologies: "java,nodejs"` — *"Ignored if the CSI volume is used or node image pull via ephemeral volume is used"*	Optional	Injection
29	Set ResourceQuotas on every namespace	`requests.cpu`, `requests.memory`, `limits.cpu`, `limits.memory`, `pods`	Recommended	Namespace
30	Set LimitRanges for default container limits	`default: {cpu: 500m, memory: 512Mi}`, `defaultRequest: {cpu: 100m, memory: 128Mi}`	Recommended	Namespace

6. Feature Flags and Annotations

#	Best Practice	Recommended Setting/Value	Priority	Category
31	Do not set the obsolete `k8s-app-enabled` flag	The docs: *"The schema is no longer available on newer Dynatrace environments, where the Kubernetes app experience is enabled automatically."*	Recommended	Feature Flag
32	Enable version detection from K8s labels	`feature.dynatrace.com/label-version-detection: "true"`	Recommended	Feature Flag
33	Set injection failure policy to `fail` in non-prod	`feature.dynatrace.com/injection-failure-policy: "fail"`	Recommended	Feature Flag
34	Set injection failure policy to `silent` in prod	`feature.dynatrace.com/injection-failure-policy: "silent"`	**Critical**	Feature Flag
35	Use opt-in for multi-tool coexistence	Namespace opt-in: `namespaceSelector` alone. Pod opt-in: `feature.dynatrace.com/automatic-injection: "false"` **plus** `oneagent.dynatrace.com/inject: "true"` on each pod — the flag alone injects nothing (K8S-11 § 2)	Recommended	Feature Flag
36	Apply `app.kubernetes.io/version` and `app.kubernetes.io/part-of` to pod templates	Read as `DT_RELEASE_VERSION` / `DT_RELEASE_PRODUCT`; only pod labels count	Recommended	Labels

#	Best Practice	Recommended Setting/Value	Priority	Category
37	Set the release stage label where you want it tracked	`dynatrace-release-stage` → `DT_RELEASE_STAGE`	Optional	Labels

7. Metadata Enrichment

#	Best Practice	Recommended Setting/Value	Priority	Category
38	Enable metadata enrichment where OneAgent does not inject	`spec.metadataEnrichment.enabled: true` (`false` by default in `v1beta6` ; OneAgent injection enriches injected pods on its own)	**Critical**	Enrichment
39	Choose the enrichment method by where the value is maintained	Central rules for existing namespace labels; `metadata.dynatrace.com/<key>` annotations for per-workload or per-pod values; DynaKube `resourceAttributes` for cluster-wide facts (K8S-10 § 2)	**Critical**	Enrichment
40	Know the precedence when two methods set the same key	Annotations > DynaKube attributes > central rules; within annotations, pod > workload > namespace	**Critical**	Enrichment
41	Create enrichment rules for `team` , `env` , `cost-center` labels	Metadata type `Label` , Source `cost-center` , Target `dt.cost.costcenter` (or "enrich directly" for a `k8s.namespace.label.<key>` field)	Recommended	Enrichment
42	Check version floors before relying on a level	Kubernetes enrichment: Operator 1.10+, OneAgent 1.333+, ActiveGate 1.343+; workload-level annotations: Operator 1.11.0+, ActiveGate 1.347+	Recommended	Enrichment
43	Wait, then restart, after rule changes	*"New rules may take up to 45 minutes to take effect. Pod restarts are required after the 45 mins"*	Recommended	Enrichment
44	Label namespaces with cost allocation metadata	`cost-center` , `budget-owner` labels on Namespace objects	Recommended	Enrichment

8. Log Monitoring

#	Best Practice	Recommended Setting/Value	Priority	Category
45	Enable log monitoring	`spec.logMonitoring: {}` , optionally with `ingestRuleMatchers` (immutable once set)	Recommended	Logs
46	Configure log monitoring resources under `templates`	`spec.templates.logMonitoring.resources: {limits: {cpu: 500m, memory: 512Mi}}`	**Critical**	Logs
47	Keep pod settings out of `spec.logMonitoring`	Its only field is `ingestRuleMatchers` ; resources and tolerations go under `spec.templates.logMonitoring`	**Critical**	Logs
48	Add control-plane tolerations to log monitoring	`spec.templates.logMonitoring.tolerations: [{effect: NoSchedule, key: node-role.kubernetes.io/control-plane, operator: Exists}]`	Recommended	Logs
49	Use OpenPipeline to filter debug logs before storage	Drop `loglevel == "DEBUG"` in processing pipeline	Recommended	Logs
50	Store logs in separate Grail buckets by environment	OpenPipeline Bucket assignment processors matching an enriched field such as `k8s.namespace.label.env`	Optional	Logs

9. OTel Collector and Telemetry Ingest

#	Best Practice	Recommended Setting/Value	Priority	Category
51	Pin OTel Collector image to a specific, verified version (example: `0.57.0`, released 09/24/2026 — check the releases before installing)	`spec.templates.otelCollector.imageRef.tag: "0.57.0"`	**Critical**	OTel
52	Never use `latest` tag for OTel Collector	Always specify explicit version tag	**Critical**	OTel
53	Set OTel Collector resource limits	`limits: {cpu: 500m, memory: 512Mi}` for staging; `{cpu: 1000m, memory: 1Gi}` for production	Recommended	OTel
54	Configure `telemetryIngest` with required protocols	`spec.telemetryIngest.protocols: [otlp]` (add `statsd`, `jaeger`, `zipkin` as needed)	Recommended	OTel
55	Do not use the OneAgent StatsD daemon on Kubernetes	Use an environment ActiveGate as remote listener (the docs' recommendation), DynaKube `telemetryIngest` with `statsd`, or a self-managed OTel Collector	**Critical**	OTel
56	If you run your own StatsD collector, give it a dedicated namespace	Cluster-wide Deployment; apps reference via FQDN `otel-collector-statsd.monitoring.svc.cluster.local:8125`	Recommended	OTel

10. CSI Driver Configuration

#	Best Practice	Recommended Setting/Value	Priority	Category
57	Decide image volumes vs. CSI vs. ephemeral volumes, then set it explicitly	See rule 7 — image volumes are recommended from Operator 1.11.0 where nodes qualify	**Critical** *(making the choice)*	CSI
58	Set resource limits on provisioner container	`provisioner.resources.limits: {cpu: 500m, memory: 256Mi}`	**Critical**	CSI
59	Set resource limits on all 5 CSI containers	`csiInit`, `server`, `provisioner`, `registrar`, `livenessprobe`	Recommended	CSI

CSI Driver Container Resource Reference

Container	Chart default (Operator 1.11.0) requests / limits	Suggested (community starting point) requests / limits
`csiInit`	50m, 100Mi / 50m, 100Mi	50m, 100Mi / 100m, 128Mi
`server`	50m, 100Mi / 50m, 100Mi	50m, 100Mi / 100m, 128Mi
`provisioner`	300m, 100Mi / **none**	300m, 100Mi / 500m, 256Mi
`registrar`	20m, 30Mi / 20m, 30Mi	20m, 30Mi / 50m, 64Mi
`livenessprobe`	20m, 30Mi / 20m, 30Mi	20m, 30Mi / 50m, 64Mi

> **Warning:** In the Operator 1.11.0 Helm chart the `provisioner` container has requests but no limits. Add them explicitly if your policies require limits.
>
> **Rules 58–59 apply only on the CSI path.** If you chose image volumes (Operator 1.11.0+) or run without the CSI driver (ephemeral volumes, see rule 7) there is no CSI DaemonSet to size, and this entire section is not applicable — the corresponding work moves to per-pod volume provisioning and pod-start latency at high density. On a Helm install the CSI driver is on unless you turned it off, so by default these rules apply.

11. Resource Sizing

#	Best Practice	Recommended Setting/Value	Priority	Category
60	Set OneAgent resources for cloudNativeFullStack	`oneAgentResources` — e.g. `requests: {cpu: 100m, memory: 256Mi}`, `limits: {cpu: 300m, memory: 512Mi}` (community starting point)	Recommended	Sizing
61	Size ActiveGate by cluster node count	See ActiveGate Sizing Reference in Section 4	**Critical**	Sizing
62	Size OTel Collector by telemetry volume (community starting points)	Low: 200m/256Mi, Medium: 500m/512Mi, High: 1000m/1Gi, Very High: 2000m/2Gi	Recommended	Sizing
63	Size Log Monitoring by daily log volume (community starting points)	Low (<1GB): 200m/256Mi, Medium (1-10GB): 500m/512Mi, High (10-50GB): 1000m/1Gi	Recommended	Sizing
64	Monitor OneAgent memory by absolute usage	OneAgent pods have limits only if you set `oneAgentResources`; use `dt.kubernetes.container.memory_working_set` with `matchesValue(k8s.container.name, "dynatrace-oneagent")`	Recommended	Sizing

12. Security and Secrets

#	Best Practice	Recommended Setting/Value	Priority	Category
65	Never store API tokens in Git	Use External Secrets Operator, Sealed Secrets, or SOPS	**Critical**	Security
66	Use External Secrets Operator for token management	`ExternalSecret` CR referencing Vault, AWS Secrets Manager, or GCP Secret Manager	Recommended	Security
67	Apply NetworkPolicies allowing Dynatrace egress on port 443	`egress: [{to: [{ipBlock: {cidr: 0.0.0.0/0}}], ports: [{protocol: TCP, port: 443}]}]`	Recommended	Security
68	Configure proxy via DynaKube spec (not env vars)	`spec.proxy.value: https://proxy.example.com:8080` or `spec.proxy.valueFrom.secretKeyRef`	Recommended	Security
69	Mirror Kubernetes namespace RBAC in Dynatrace IAM	A namespace bound to `view` / `edit` / `admin` maps to a read policy with `WHERE storage:k8s.namespace.name = "<ns>"` (K8S-06 § 4)	Recommended	Security

13. GitOps and Lifecycle

#	Best Practice	Recommended Setting/Value	Priority	Category
70	Manage DynaKube via GitOps (ArgoCD or Flux)	Store DynaKube YAML in Git, apply via GitOps controller	Recommended	Lifecycle
71	Use Kustomize overlays for environment-specific config	`base/dynakube.yaml` + `overlays/{dev,staging,prod}/` patches	Recommended	Lifecycle
72	Pin an **exact** operator chart version in ArgoCD/Flux	`targetRevision: 1.10.2` (ArgoCD) or `version: "1.10.2"` (Flux) — prefer an exact pin over a `1.10.x` range, because an operator upgrade can change chart *defaults* with no Git commit to review (K8S-02 §8). Never pin below 1.4.1; skip 1.10.0	**Critical**	Lifecycle
73	Set `selfHeal: true` in ArgoCD for drift correction	`syncPolicy.automated.selfHeal: true`	Recommended	Lifecycle
74	Set `prune: false` for DynaKube in ArgoCD	Prevent accidental DynaKube deletion on Git removal	**Critical**	Lifecycle

#	Best Practice	Recommended Setting/Value	Priority	Category
75	Use Flux `dependsOn` to order operator before DynaKube	`dependsOn: [{name: dynatrace-operator}]`	Recommended	Lifecycle
76	Use ArgoCD sync waves for deployment ordering	Wave 0: Namespace, Wave 1: Operator, Wave 2: DynaKube	Recommended	Lifecycle
77	Use `helm upgrade --reuse-values` for operator upgrades — **and diff the rendered chart**	Preserves *your* values, but **not** changed chart *defaults* for values you never set. Render and diff before upgrading (`helm template ... > new.yaml` vs. `helm get manifest`) — K8S-02 §8	Recommended	Lifecycle
78	Deploy to dev -> staging -> prod-canary -> prod	Progressive rollout strategy	Recommended	Lifecycle

14. Cluster Health Alerting

#	Best Practice	Recommended Setting/Value	Priority	Category
79	Alert on Node NotReady	Built-in *Detect node readiness issues*, or `dt.kubernetes.node.conditions` Ready = false for >5 min = Critical	**Critical**	Alerting
80	Alert on high node CPU	CPU > 85% for 15 min = Warning	Recommended	Alerting
81	Alert on high node memory	Memory > 90% for 10 min = Critical	**Critical**	Alerting
82	Alert on disk pressure	DiskPressure condition true, or disk > 85% = Warning	Recommended	Alerting
83	Alert on OOM kills	`dt.kubernetes.container.oom_kills` > 0 = Warning (a metric, not an event)	**Critical**	Alerting
84	Alert on restart loops	`BackOff` events or `dt.kubernetes.container.restarts` = Warning (CrashLoopBackOff is message text, not a reason)	Recommended	Alerting
85	Alert on FailedScheduling	Events > 5 in 10 min = Warning	Recommended	Alerting
86	Alert on volume mount failures	Any FailedMount or FailedAttachVolume = Critical	**Critical**	Alerting
87	Monitor Dynatrace component health (OA, AG, CSI)	Track restarts, OOMKills, and data gaps for `dynatrace-oneagent` and `activegate` containers	**Critical**	Alerting
88	Right-size CPU requests	Workload CPU used / requested; ~40% average is a community starting point (K8S-04 § 6)	Recommended	Cost
89	Right-size memory requests	Workload working set / requested; ~50% average is a community starting point, keep peak headroom	Recommended	Cost

15. Workload and Application Monitoring

#	Best Practice	Recommended Setting/Value	Priority	Category
90	Alert on container CPU usage > 85% sustained	Monitor `dt.kubernetes.container.cpu_usage` vs limits	Recommended	Workload
91	Alert on container memory usage > 90%	Monitor `dt.kubernetes.container.memory_working_set` vs limits	**Critical**	Workload
92	Monitor CPU throttling	`dt.containers.cpu.throttled_time > 0` indicates under-provisioned CPU limits	Recommended	Workload
93	Track service P99 latency and error rate	SLO: P99 < 500ms, error rate < 0.1%	Recommended	Workload

#	Best Practice	Recommended Setting/Value	Priority	Category
94	Filter on `isNotNull(field)` before aggregating optional fields	Prevent null group-by keys in DQL	Recommended	Workload
95	Use `arrayAvg()` before filtering/sorting timeseries results	`timeseries` returns arrays; convert to scalar first	**Critical**	DQL

16. Specialized Monitoring

#	Best Practice	Recommended Setting/Value	Priority	Category
96	Instrument NGINX Ingress via ConfigMap `main-snippet`	`load_module /opt/dynatrace/oneagent-paas/agent/bin/current/linux-musl-x86-64/liboneagentnginx.so;`	Recommended	NGINX
97	NGINX instrumentation requires OneAgent 1.227+ and no ARM64	Pod or container name must contain `ingress-nginx-` or `nginx-ingress-`	Recommended	NGINX
98	Enable Prometheus scraping for custom metrics	Annotate pods with `metrics.dynatrace.com/scrape: "true"`, `/port`, `/path`	Recommended	Prometheus
99	Scrape Prometheus from inside the cluster — and prefer the OTel Collector for new setups	An external ActiveGate cannot scrape endpoints that require authentication; the ActiveGate integration has hard limits (1,000 exporter pods, 1,000 metrics per pod), and the docs recommend the OpenTelemetry Collector for new deployments	**Critical**	Prometheus
100	Use Kpow + Prometheus scraping for Kafka consumer lag monitoring	`PROMETHEUS_EGRESS: "true"` in Kpow config; scrape annotations on Kpow pods	Optional	Kafka

17. Troubleshooting Practices

#	Best Practice	Recommended Setting/Value	Priority	Category
101	Check DynaKube status first	`kubectl -n dynatrace get dynakube` — expect `Running` phase	**Critical**	Troubleshooting
102	Verify all Dynatrace pods are running	`kubectl -n dynatrace get pods` — no CrashLoopBackOff or Error	**Critical**	Troubleshooting
103	Check OneAgent connectivity to tenant	`kubectl -n dynatrace exec <oneagent-pod> -c oneagent -- curl -v https://<tenant>.live.dynatrace.com/api/v1/time`	Recommended	Troubleshooting
104	Check namespace labels when injection fails	`kubectl get namespace <ns> -o jsonpath='{.metadata.labels}'` must match DynaKube `namespaceSelector`	Recommended	Troubleshooting
105	Verify init container present for injected pods	`kubectl get pod <pod> -o jsonpath='{.spec.initContainers[*].name}'`	Recommended	Troubleshooting
106	Check webhook registration	`kubectl get mutatingwebhookconfigurations` — Dynatrace webhook must exist	Recommended	Troubleshooting
107	Collect a support archive for escalation	`kubectl exec -n dynatrace deployment/dynatrace-operator -- dynatrace-operator support-archive --stdout > operator-support-archive.zip` (K8S-09 § 8)	Optional	Troubleshooting

#	Best Practice	Recommended Setting/Value	Priority	Category
108	Use DQL to detect Dynatrace component failures across fleet	`events` with `event.provider == "KUBERNETES_EVENT"` in the `dynatrace` namespace (`Failed` , `BackOff` , `FailedMount`), plus the `oom_kills` and `restarts` metrics	Recommended	Troubleshooting
109	Detect metric data gaps from null buckets	`timeseries … = avg(dt.host.cpu.usage)` then compare `arraySize(arrayRemoveNulls(…))` to `arraySize(…)` — `arraySize` counts empty buckets and `arrayMin` skips them	Recommended	Troubleshooting

---

Summary

This notebook contains **109 actionable best practices** across 17 categories for Dynatrace Kubernetes monitoring:

Category	Count	Key Takeaway
Deployment Mode	5	Use `cloudNativeFullStack` ; never `classicFullStack` for new deployments
Operator Installation	5	Helm OCI install with an explicit version; choose code-module delivery deliberately
DynaKube Core	7	`v1beta6` , tolerations, resource limits
ActiveGate	5	`kubernetes-monitoring` + `routing` , 2+ replicas in prod
Namespace/Injection	8	`namespaceSelector` for scope control, ResourceQuotas on every namespace
Feature Flags	7	Version detection, opt-in levels, injection failure policy
Metadata Enrichment	7	Three methods, one precedence order, version floors
Log Monitoring	6	`logMonitoring` (optionally `ingestRuleMatchers`), resources under `templates`
OTel/Telemetry Ingest	6	Pin version; StatsD via environment ActiveGate, telemetry ingest or a collector
CSI Driver	3	Provisioner has no default limits; set them on the CSI path
Resource Sizing	5	Size by cluster/volume tier
Security	5	Never store tokens in Git, use ESO
GitOps/Lifecycle	9	Pin versions, progressive rollouts, prune: false for DynaKube
Cluster Alerting	11	Node health, OOMKill, CrashLoop, Dynatrace component health
Workload Monitoring	6	CPU throttling, P99 SLOs, `arrayAvg()` for timeseries
Specialized Monitoring	5	NGINX via ConfigMap; Prometheus from inside the cluster, OTel Collector for new setups
Troubleshooting	9	DynaKube status, connectivity, DQL-based diagnostics

---

References

- DynaKube parameters (DT docs)
- DynaKube feature flags (DT docs)
- Set up Dynatrace on Kubernetes (DT docs)
- How Dynatrace Kubernetes monitoring works (DT docs)
- Quickstart for K8s setup (DT docs)
- Full observability deployment (DT docs)
- Kubernetes log monitoring (DT docs)

- Metadata enrichment (DT docs)
- Kubernetes app (DT docs)
- Operator + DynaKube troubleshooting (DT docs)
- Dynatrace Operator Helm chart (Dynatrace GitHub)
- Dynatrace Operator releases (Dynatrace GitHub)
- Operator 1.11.0 release notes (DT docs)
- Kubernetes tag setup (DT docs)

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.